

DS4300 HW1 — Twitter on MySQL: Performance Report

BingXian (Eason) Xie | DS4300, Prof. Rachlin | Summer A 2026

1. Hardware

Machine	Apple MacBook Air (Mac15,12)
CPU	Apple M3 SoC, 8 cores (4 performance + 4 efficiency)
RAM	16 GiB unified memory (LPDDR5)
Storage	245 GB internal Apple NVMe SSD, APFS (Apple Fabric protocol)
Network	Loopback only — MySQL and the JVM run on the same machine

2. Software Stack

OS	macOS 14.6 (build 23G80)
Database	MySQL 9.6.0 (Homebrew innovation release)
JVM	Oracle Java 21.0.8 LTS (HotSpot 64-bit Server VM, build 21.0.8+12-LTS-250)
Build tool	Apache Maven 3.9.11
JDBC driver	<code>com.mysql:mysql-connector-j:9.6.0</code>
CSV parser	<code>com.opencsv:opencsv:5.12.0</code>
Engine / runtime	InnoDB, autocommit on, one <code>Connection</code> per bench, no pool
JDBC URL flags	<code>rewriteBatchedStatements=false, useServerPrepStmts=true, serverTimezone=UTC</code>

3. Results

Two cached `PreparedStatement`s drive both numbers: one `INSERT` (`postTweet`) and one `SELECT ...JOIN` follows `...ORDER BY tweet_ts DESC LIMIT 10` (`getTimeline`). `postTweet` streams **1,000,000** rows from `tweet.csv` with one `executeUpdate` per row (no batching). `getTimeline` runs **10,000** timed iterations after a **1,000**-iteration JIT warm-up, sampled uniformly from the 9,469 distinct `follower_ids` in `follows`.

API function	Calls/sec	Twitter target	Verdict
<code>postTweet</code>	7,383.1	6,000 – 10,000	in range
<code>getTimeline</code>	1,450.4	200,000 – 300,000	~140× short

`postTweet`: 1M rows in 135.44 s; rolling window jitter 6.5–8.9k/s after warm-up. `getTimeline`: 10k calls in 6.895 s; every call returned exactly 10 tweets (100k rows materialized into `List<Tweet>`).

4. Factors That Impacted the Numbers

- **Indexing is decisive for `getTimeline`.** `idx_user_ts` (`user_id`, `tweet_ts` DESC) on `tweet` plus `idx_follower` on `follows` let the JOIN do a ref lookup on `follows` followed by an indexed range scan per followee, instead of a full table scan over 1M tweet rows. EXPLAIN confirms both indexes are used.
- **Per-row inserts dominate `postTweet` latency, by design.** With `rewriteBatchedStatements=false` the driver cannot fold multi-row INSERTs, so every tweet incurs one loopback round-trip plus an InnoDB redo-log fsync. Autocommit means each insert is its own transaction. Both are deliberate, per the PDF’s “one at a time, no batching” rule.
- **No connection pool and no per-row I/O.** A single `Connection` is held for the bench, and all progress logging is buffered in two `long[]` arrays and flushed *after* the timed window closes. This isolates the measurement from pool warm-up and from `println` jitter. JIT warm-up matters as well: the first ~200 untimed `getTimeline` calls measured 30–40% slower than the timed ones because HotSpot was still interpreting `ResultSet` unpacking.
- **Local SSD and laptop noise.** The Apple NVMe SSD sustains ≥ 1 GB/s writes, so disk bandwidth is not the bottleneck for the redo log — per-row fsync syscall cost is. The run was on a development laptop with a browser, IDE, and Spotlight active; that accounts for most of the window-by-window variance in `post_tweet_bench.csv`.

5. Use of AI

(i) *Debugging.* When JDBC, Maven, or MySQL errors stopped progress, I described what i was doing and copy the log error to the assistant and used its hypotheses as a starting point. (ii) *Dependency selection.* I asked the assistant to recommend known-good versions of `mysql-connector-j` and `opencsv` for Java 21, because a prior project had taught me how expensive Maven version-resolution mistakes can be. I did not delegate the design of the `TwitterAPI` seam, the choice of indexes, the SQL, or the benchmark methodology.